

Dockerfile FastAPI - Explicacion y Comandos Utiles

Guia practica para entender, construir, ejecutar, probar y depurar una aplicacion FastAPI empaquetada en Docker.

1. Dockerfile comentado

```
# Imagen base oficial de Python.
# La variante slim es mas liviana que la imagen completa.
FROM python:3.13-slim

# Directorio de trabajo dentro del contenedor.
WORKDIR /app

# Evita generar archivos .pyc y carpetas __pycache__.
ENV PYTHONDONTWRITEBYTECODE=1

# Permite ver logs en tiempo real con docker logs.
ENV PYTHONUNBUFFERED=1

# Instala curl y limpia cache de apt para reducir peso.
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Copia primero dependencias para aprovechar cache de Docker.
COPY requirements.txt .

# Instala dependencias Python sin cache.
RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir -r requirements.txt

# Copia el codigo completo de la aplicacion.
COPY . .

# Elimina __pycache__ que venga desde el host.
RUN find . -type d -name "__pycache__" -exec rm -rf {} + || true

# Documenta que la app escucha en el puerto 8000.
EXPOSE 8000

# Levanta FastAPI con Uvicorn.
# app.main:app = archivo app/main.py, variable app.
# 0.0.0.0 permite acceso desde fuera del contenedor.
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2. Build y ejecucion

```
# Construir imagen
docker build -t fastapi-sqlite-demo .

# Ejecutar en primer plano
docker run --name fastapi-demo -p 8000:8000 fastapi-sqlite-demo

# Ejecutar en segundo plano
docker run -d --name fastapi-demo -p 8000:8000 fastapi-sqlite-demo

# Ver contenedores activos
docker ps

# Ver logs en vivo
docker logs -f fastapi-demo

# Detener y eliminar
docker stop fastapi-demo
docker rm fastapi-demo
```

3. Probar la API desde navegador

Swagger UI:
`http://localhost:8000/docs`

Redoc:
`http://localhost:8000/redoc`

OpenAPI JSON:
`http://localhost:8000/openapi.json`

4. Pruebas con curl

```
# GET simple
curl -i http://localhost:8000/

# Listar usuarios, si existe ese endpoint
curl -i http://localhost:8000/users

# Crear usuario
curl -i -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Juan","email":"juan@example.com"}'

# Consultar usuario por ID
curl -i http://localhost:8000/users/1

# Actualizar usuario, si tu API usa PUT
curl -i -X PUT http://localhost:8000/users/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"Juan Actualizado","email":"juan.actualizado@example.com"}'

# Eliminar usuario
curl -i -X DELETE http://localhost:8000/users/1

# Formatear OpenAPI JSON
curl -s http://localhost:8000/openapi.json | python -m json.tool
```

5. Pruebas con HTTPie

```
# Instalar HTTPie
python -m pip install --user httpie

# GET
http GET :8000/users

# POST
http POST :8000/users name="Juan" email="juan@example.com"

# GET por ID
http GET :8000/users/1

# DELETE
http DELETE :8000/users/1
```

6. Entrar al contenedor

```
# Entrar con sh, porque python:slim normalmente no trae bash
docker exec -it fastapi-demo sh

# Ver archivos
ls -lah /app

# Ver version Python
python --version

# Ver dependencias
```

```
pip list
```

```
# Probar import de la app
python -c "from app.main import app; print(app.title)"
```

7. SQLite y persistencia

```
# Si app.db queda dentro del contenedor, se pierde al eliminarlo.
# Montar archivo SQLite desde el host:
```

```
docker run -d --name fastapi-demo \
  -p 8000:8000 \
  -v "$PWD/app.db:/app/app.db" \
  fastapi-sqlite-demo
```

```
# Alternativa: montar carpeta completa de datos
mkdir -p ./data
docker run -d --name fastapi-demo \
  -p 8000:8000 \
  -v "$PWD/data:/app/data" \
  fastapi-sqlite-demo
```

8. Diagnostico y limpieza

```
# Consumo de recursos
docker stats fastapi-demo
```

```
# Inspeccionar configuracion
docker inspect fastapi-demo
```

```
# Procesos dentro del contenedor
docker exec -it fastapi-demo ps aux
```

```
# Reiniciar
docker restart fastapi-demo
```

```
# Limpiar contenedores detenidos
docker container prune
```

```
# Limpiar imagenes sin uso
docker image prune
```

```
# Limpiar todo lo no usado, usar con cuidado
docker system prune -a
```

9. Recomendaciones practicas

- Crear `.dockerignore` para excluir `.venv`, `.git`, `__pycache__`, archivos temporales y datos pesados.
- No guardar secretos dentro de la imagen. Usar variables de entorno o gestor de secretos.
- Para produccion real con alta concurrencia, preferir PostgreSQL sobre SQLite.
- Agregar HEALTHCHECK si el orquestador o plataforma lo aprovecha.
- Para multiples workers, evaluar Gunicorn con UvicornWorker.